Solution: Missing Ranges

Problem Statement

You are given two integers, `lower` and `upper`, specifying the inclusive range, and the sorted array of unique integers `nums`, where each element is in between `[lower, upper]`.

We can say the number `n` is missing if `n` is not in the `nums` and falls in the range `[lower, upper]`.

Return the `shortest sorted` list of ranges that `covers all missing numbers`. These ranges should fall within the inclusive bounds specified by the lower and upper limit integers.

Examples

Example 1:

- **Input**: `nums = [3, 5, 10, 20]`, `lower = 1`, `upper = 22`
- **Expected Output**: `[[1, 2], [4, 4], [6, 9], [11, 19], [21, 22]]`
- **Justification**: The missing ranges are 1-2, 4, 6-9, 11-19, and 21-22. Each missing range is represented as a pair in the list, where both numbers are the same for a single missing number.

Example 2:

- **Input**: `nums = [10, 20, 30, 40, 50]`, `lower = 5`, `upper = 55`
- **Expected Output**: `[[5, 9], [11, 19], [21, 29], [31, 39], [41, 49], [51, 55]]`
- **Justification**: The missing ranges are 5-9, 11-19, 21-29, 31-39, 41-49, and 51-55. Each interval is represented as a pair of start and end values.

Example 3:

- **Input**: `nums = [1, 3, 6, 10]`, `lower = 0`, `upper = 12`
- **Expected Output**: `[[0, 0], [2, 2], [4, 5], [7, 9], [11, 12]]`
- **Justification**: The missing ranges are 0, 2, 4-5, 7-9, and 11-12. Each range is represented as a start and end pair.

Solution

To solve this problem, the primary approach involves iterating through the list and identifying gaps between consecutive elements compared to the given lower and upper bounds. The solution must account for ranges at the beginning and end of the list that might not be covered by the elements.

This approach works effectively because it systematically checks each interval between the numbers in the sorted list against the specified bounds. By comparing adjacent numbers and the bounds, we can easily identify the missing ranges. The algorithm is simple and efficient for a sorted list, as it requires only a single pass through the list.

Step-by-step Algorithm

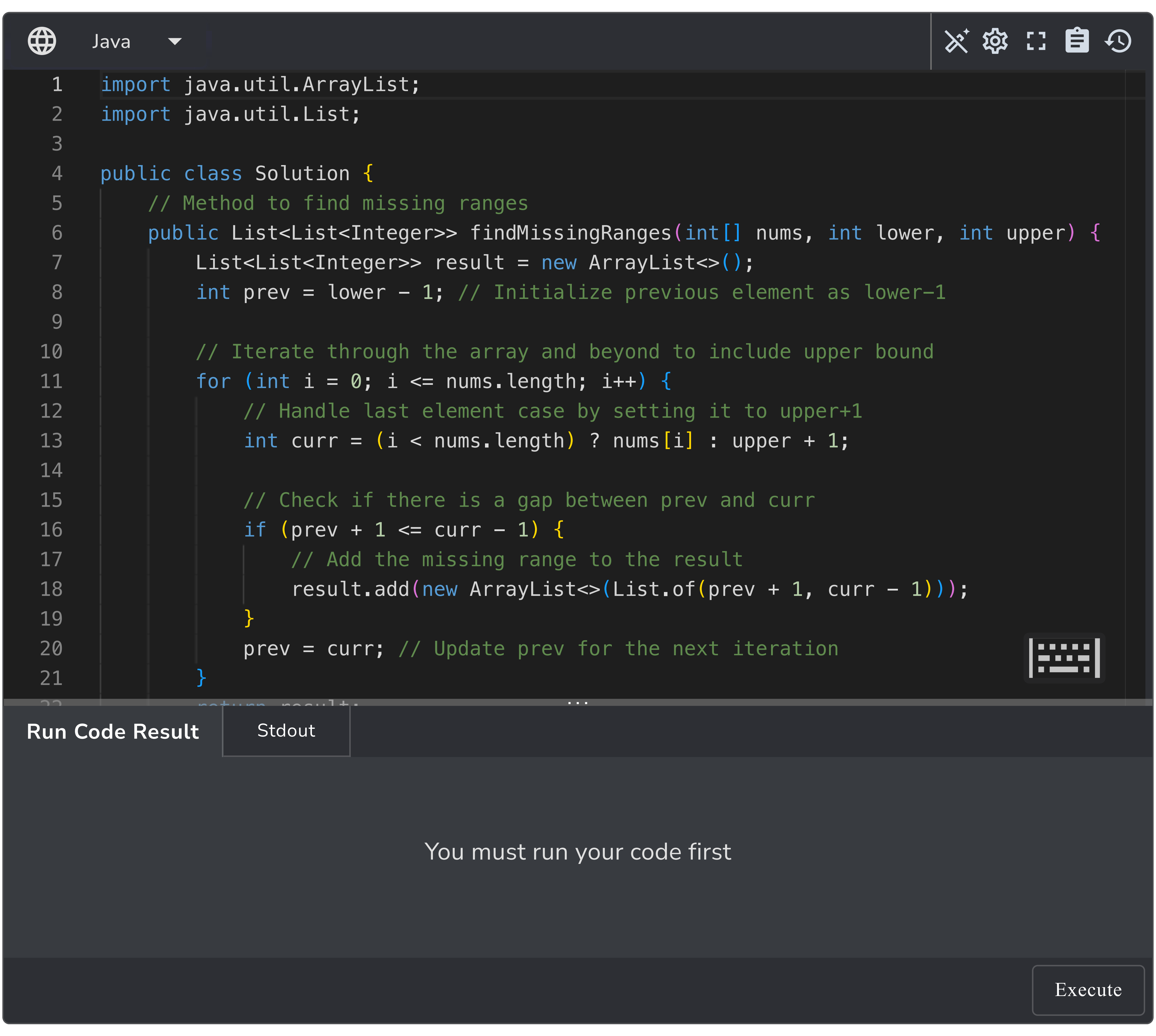
- Initialize an empty list `result` to store the missing ranges.
- Add the range from `lower` to the first element in the list (if needed).
- Iterate through the list of numbers:
  - For each pair of adjacent numbers, check if there is a gap (difference > 1).
  - If a gap exists, add the missing range to `result`.
- After the loop, add the range from the last element to `upper` (if needed).
- Return the `result` list containing all the missing ranges.

Algorithm Walkthrough

Let's use **Example 1** for the walkthrough:

- **Input**: `nums = [3, 5, 10, 20]`, `lower = 1`, `upper = 22`
- **Steps**:
  1. Initialize `result` as an empty list.
  2. The first number (3) does not match `lower` (1), so we add the range `[1, 2]`.
  3. Check gaps:
     - Between 3 and 5, gap exists (4), add `[4, 4]`.
     - Between 5 and 10, gap exists (6-9), add `[6, 9]`.
     - Between 10 and 20, gap exists (11-19), add `[11, 19]`.
  4. After the loop, add the range from the last number (20) to `upper` (22), i.e., `[21, 22]`.
  5. The final `result` is `[[1, 2], [4, 4], [6, 9], [11, 19], [21, 22]]`.

Code



```java
import java.util.ArrayList;
import java.util.List;

public class Solution {
    // Method to find missing ranges
    public List<List<Integer>> findMissingRanges(int[] nums, int lower, int upper) {
        List<List<Integer>> result = new ArrayList<>();
        int prev = lower - 1; // Initialize previous element as lower-1

        // Iterate through the array and beyond to include upper bound
        for (int i = 0; i <= nums.length; i++) {
            // Handle last element case by setting it to upper+1
            int curr = (i < nums.length) ? nums[i] : upper + 1;

            // Check if there is a gap between prev and curr
            if (prev + 1 <= curr - 1) {
                // Add the missing range to the result
                result.add(new ArrayList<>(List.of(prev + 1, curr - 1)));
            }
            prev = curr; // Update prev for the next iteration
        }
```

You must run your code first

Complexity Analysis

Time Complexity: (O(N))

The algorithm iterates through the given list of (N) elements once. Each iteration involves constant-time operations. Therefore, the overall time complexity is linear, (O(N)).

Space Complexity: (O(M))

The space complexity is determined by the number of missing ranges stored in the result list. In the worst case, this could be proportional to the difference between `upper` and `lower` bounds. If (M) represents the number of missing ranges, the space complexity is (O(M)).

← Previous
Missing Ranges (Easy)
Next →
Palindrome Permutation (Easy)
☐ Mark as Completed